

Day 20 – Kubernetes & Helm for FastAPI + React GenAI Apps

1. Learning objective

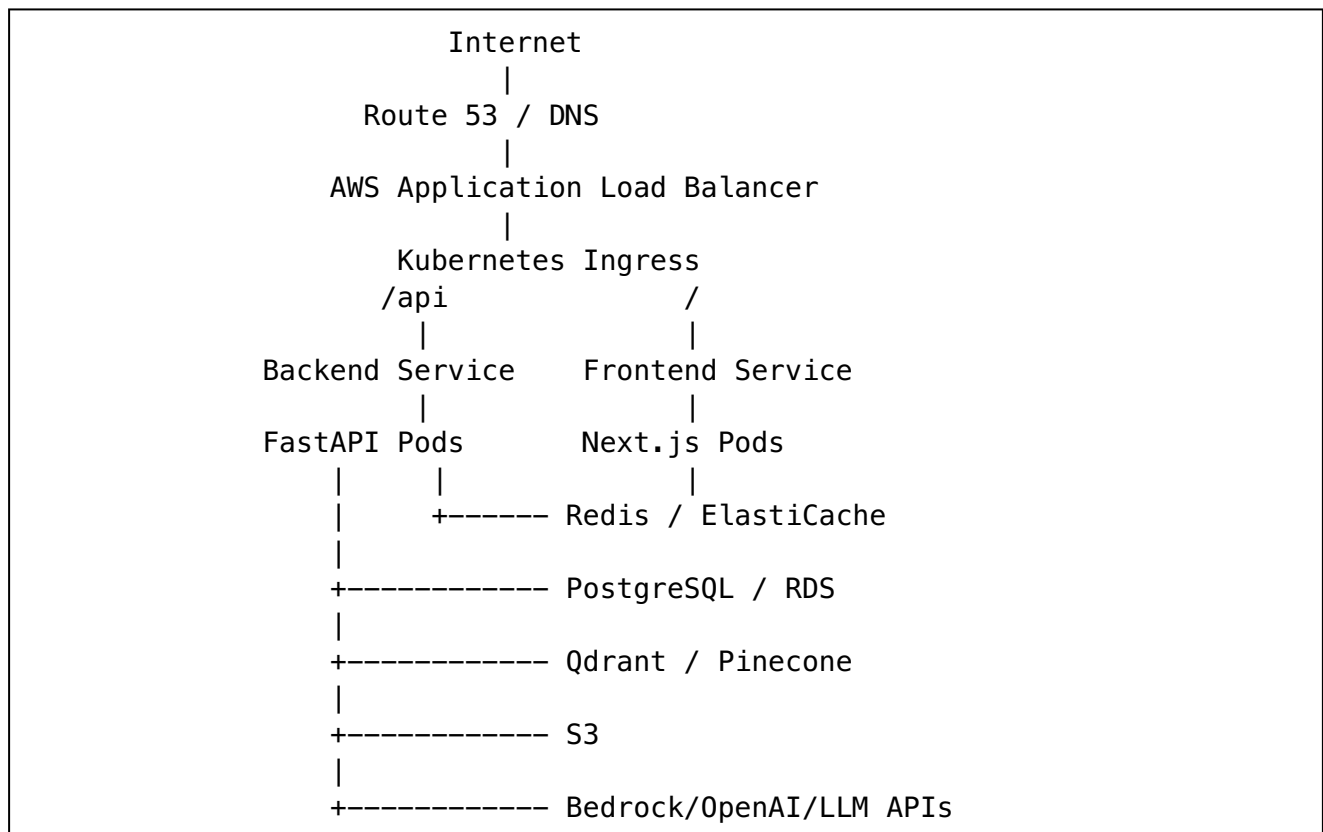
Today you will understand how to package and deploy this capstone architecture on Amazon EKS:

- **FastAPI backend** serving RAG and agent APIs
- **React/Next.js frontend** serving the chat interface
- **PostgreSQL/RDS** for application data
- **Redis/ElastiCache** for caching, sessions and job state
- **Qdrant/Pinecone** for vector search
- **S3** for uploaded documents
- **External LLM APIs** or a separately hosted inference service
- **Helm** for repeatable Kubernetes deployments

The main idea is:

Terraform creates the EKS cluster and cloud infrastructure. Helm deploys and manages the application workloads inside Kubernetes.

2. Capstone architecture



On EKS, the AWS Load Balancer Controller watches Kubernetes Ingress resources and can provision an AWS Application Load Balancer. The ALB can then route /api requests to the backend Service and / requests to the frontend Service. ([AWS Documentation](#))

3. Kubernetes recap for a GenAI application

3.1 Pod

A **Pod** is the smallest deployable Kubernetes compute object. It normally contains one application container, although it can contain sidecars.

For this project:

```
Backend Pod
└─ FastAPI container

Frontend Pod
└─ Next.js container
```

You generally do not create individual Pods directly. You create a Deployment, which creates and manages the Pods.

Pods are temporary. A Pod can be replaced, rescheduled or assigned a new IP address, so applications should not depend on a particular Pod identity.

3.2 Deployment

A **Deployment** describes the desired application state:

- Container image
- Number of replicas
- CPU and memory configuration
- Environment variables
- Health probes
- Update strategy

Example:

```
FastAPI Deployment
├─ FastAPI Pod 1
├─ FastAPI Pod 2
└─ FastAPI Pod 3
```

If one Pod fails, the Deployment controller creates a replacement.

Deployments also support rolling updates, allowing old and new application versions to run temporarily during a rollout. ([Kubernetes](#))

3.3 Service

Pod IP addresses are not stable. A Kubernetes **Service** provides a stable name and virtual endpoint in front of a changing group of Pods.

For example:

```
genai-backend-service
|
+-- backend-pod-1
```

```
+--- backend-pod-2
+--- backend-pod-3
```

The frontend or Ingress does not need to know which backend Pod is currently available.

Inside the cluster, the backend might be available through DNS as:

```
genai-backend.genai-prod.svc.cluster.local
```

A Service generally selects Pods using labels. Kubernetes updates the endpoints behind the Service as Pods are created or removed. ([Kubernetes](#))

For this architecture, backend and frontend Services should usually be:

```
type: ClusterIP
```

They remain private inside Kubernetes. The Ingress becomes the public entry point.

3.4 Ingress

An **Ingress** contains HTTP routing rules.

For the capstone:

```
https://chat.example.com/api/* → FastAPI Service
https://chat.example.com/*      → Next.js Service
```

An Ingress resource does not route traffic by itself. An Ingress controller must watch and implement it. Every Ingress should identify the appropriate Ingress class. ([Kubernetes](#))

On EKS, a common option is:

```
ingressClassName: alb
```

The AWS Load Balancer Controller then creates and configures an AWS ALB.

3.5 ConfigMap

A **ConfigMap** stores non-sensitive application configuration.

Examples:

- Environment name
- Log level
- Redis cache TTL
- Vector collection name
- Model name
- S3 bucket name
- Feature flags
- Maximum upload size
- RAG retrieval count

Example:

```
APP_ENV=production
LOG_LEVEL=INFO
TOP_K=5
S3_BUCKET=genai-prod-documents
```

A ConfigMap should not contain API keys, passwords or database credentials.

3.6 Secret

A Kubernetes **Secret** holds sensitive configuration such as:

- Database username and password
- Database URL
- Redis authentication token
- Pinecone API key
- OpenAI API key
- JWT signing key

Important:

| A Kubernetes Secret is not automatically a complete secrets-management solution.

The application should avoid storing plaintext secret values in:

- Git
- `values.yaml`
- CI logs
- Terraform outputs
- Helm command history

A production implementation would commonly combine Kubernetes with:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- Secrets Store CSI Driver
- External Secrets Operator

For your learning architecture, Terraform can conceptually create or synchronize a Kubernetes Secret, and the Helm chart can reference the existing Secret by name.

4. Why Kubernetes is useful for RAG and agent services

Independent scaling

The frontend may need only two replicas, while the FastAPI RAG backend may need ten replicas during high traffic.

You can scale them independently:

```
Frontend: 2 Pods
Backend:  8 Pods
Worker:   4 Pods
```

Failure recovery

If a FastAPI container crashes because of an out-of-memory error or application failure, Kubernetes can replace it.

Controlled deployments

Kubernetes supports:

- Rolling updates
- Rollback
- Readiness checks
- Liveness checks
- Canary patterns
- Blue/green patterns

Resource management

You can configure different CPU and memory requirements:

```
Frontend Pod: 250m CPU, 256 MiB
Backend Pod:  1 CPU, 2 GiB
Worker Pod:   2 CPU, 4 GiB
```

Environment separation

The same chart can be deployed as:

```
genai-dev
genai-stage
genai-prod
```

Each environment supplies different Helm values.

Service discovery

The frontend, backend and internal workers communicate using Kubernetes Service names rather than changing Pod IP addresses.

Scaling limitations to remember

CPU-based scaling may work for ordinary FastAPI workloads, but RAG systems are often I/O-bound because they wait for:

- Vector database queries
- LLM responses
- Reranking APIs
- Database queries

For production, useful scaling metrics may include:

- Active requests
- Requests per second
- Pending jobs
- Event-loop latency

- LLM request concurrency
- Queue depth
- P95 response latency

Kubernetes HPA automatically changes workload replica counts based on configured metrics. CPU and memory are common starting points, but custom metrics are often more appropriate for GenAI services. ([Kubernetes](#))

5. Helm basics

Helm is a package manager and templating system for Kubernetes applications.

Without Helm, you might maintain separate YAML files such as:

```
backend-deployment-dev.yaml
backend-deployment-stage.yaml
backend-deployment-prod.yaml
backend-service-dev.yaml
backend-service-stage.yaml
...
```

This becomes repetitive.

With Helm, you create reusable templates:

```
replicas: {{ .Values.replicaCount }}
```

Then supply environment-specific values.

5.1 Helm chart

A **chart** is a directory containing:

- Kubernetes YAML templates
- Default values
- Chart metadata
- Optional dependencies

A chart commonly has this structure:

```
genai-backend/
├── Chart.yaml
├── values.yaml
├── values-dev.yaml
├── values-prod.yaml
├── templates/
│   ├── _helpers.tpl
│   ├── configmap.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── hpa.yaml
│   └── serviceaccount.yaml
```

Helm renders files from `templates/`, injects the selected values, and sends the resulting Kubernetes resources to the cluster. `Chart.yaml` describes the chart, while `values.yaml` provides its default configuration. (helm.sh)

5.2 Recommended capstone chart organization

Use separate charts because backend and frontend have different:

- Scaling requirements
- Container images
- Health endpoints
- Release schedules
- Resource requirements
- Configuration

```
deploy/  
└─ helm/  
    └─ genai-backend/  
        └─ Chart.yaml  
        └─ values.yaml  
        └─ values-prod.yaml  
        └─ templates/  
    └─ genai-frontend/  
        └─ Chart.yaml  
        └─ values.yaml  
        └─ values-prod.yaml  
        └─ templates/  
    └─ genai-routing/  
        └─ Chart.yaml  
        └─ values.yaml  
        └─ templates/  
            └─ ingress.yaml
```

Example releases:

```
genai-backend-prod  
genai-frontend-prod  
genai-routing-prod
```

This allows you to roll back only the backend without changing the frontend.

For a smaller project, the Ingress could instead live in the frontend chart.

5.3 Chart.yaml

```
apiVersion: v2  
name: genai-backend  
description: FastAPI RAG and agent backend  
type: application
```

```
version: 0.1.0
appVersion: "1.4.2"
```

There are two versions to understand:

```
version      → Version of the Helm chart
appVersion   → Version of the application represented by the chart
```

The image tag should normally come from values rather than being inferred from appVersion.

5.4 Values

Example values.yaml:

```
replicaCount: 2

image:
  repository: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/genai-backend
  tag: "1.4.2"
  pullPolicy: IfNotPresent

service:
  port: 80
  targetPort: 8000

config:
  appEnv: production
  logLevel: INFO
  vectorDbUrl: https://qdrant.internal.example.com
  vectorCollection: product-documents
  topK: "5"

existingSecretName: genai-backend-secrets

resources:
  requests:
    cpu: 500m
    memory: 1Gi
  limits:
    cpu: "2"
    memory: 3Gi

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 65
```

Values can come from:

1. Default values.yaml
2. An environment values file supplied with `-f`
3. Individual `--set` arguments

More specific sources override less specific sources. ([helm.sh](#))

Example:

```
helm upgrade --install genai-backend-prod \  
  ./deploy/helm/genai-backend \  
  --namespace genai-prod \  
  --create-namespace \  
  -f ./deploy/helm/genai-backend/values-prod.yaml \  
  --set image.tag=1.4.3
```

6. Important Helm template objects

.Values

Reads configuration from Helm values:

```
replicas: {{ .Values.replicaCount }}
```

```
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

.Release.Name

Provides the Helm release name:

```
metadata:  
  name: {{ .Release.Name }}-backend
```

Installing the chart with:

```
helm install genai-prod ./genai-backend
```

produces a resource name such as:

```
genai-prod-backend
```

.Release.Namespace

Provides the target namespace:

```
metadata:  
  namespace: {{ .Release.Namespace }}
```

Usually, namespace fields can be omitted and Helm will install resources in the release namespace.

quote

Quotes a value safely:

```
value: {{ .Values.config.logLevel | quote }}
```

toYaml and nindent

Useful for inserting nested YAML:

```
resources:
{{- toYaml .Values.resources | nindent 10 }}
```

7. Backend chart design

The FastAPI backend requires:

- Deployment
- ClusterIP Service
- ConfigMap
- Existing Secret reference
- Readiness and liveness probes
- Resource requests and limits
- Optional HPA
- Service account if it accesses AWS services using IAM

The container should listen on:

```
0.0.0.0:8000
```

For example:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

8. Helm template example 1: ConfigMap and FastAPI Deployment

```
# templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-backend-config
data:
  APP_ENV: {{ .Values.config.appEnv | quote }}
  LOG_LEVEL: {{ .Values.config.logLevel | quote }}
  VECTOR_DB_URL: {{ .Values.config.vectorDbUrl | quote }}
  VECTOR_COLLECTION: {{ .Values.config.vectorCollection | quote }}
  RAG_TOP_K: {{ .Values.config.topK | quote }}

---
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-backend
spec:
```

```

replicas: {{ .Values.replicaCount }}
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0

selector:
  matchLabels:
    app: {{ .Release.Name }}-backend

template:
  metadata:
    labels:
      app: {{ .Release.Name }}-backend

  spec:
    containers:
      - name: fastapi
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}

        ports:
          - name: http
            containerPort: 8000

        envFrom:
          - configMapRef:
              name: {{ .Release.Name }}-backend-config

          - secretRef:
              name: {{ .Values.existingSecretName }}

        readinessProbe:
          httpGet:
            path: /health/ready
            port: http
            initialDelaySeconds: 10
            periodSeconds: 10

        livenessProbe:
          httpGet:
            path: /health/live
            port: http
            initialDelaySeconds: 30
            periodSeconds: 20

        resources:
{{- toYaml .Values.resources | nindent 12 }}

```

Expected Secret keys

The existing Secret could contain:

```
DATABASE_URL
REDIS_URL
PINECONE_API_KEY
OPENAI_API_KEY
JWT_SECRET
```

For example, Terraform conceptually creates:

```
Secret name: genai-backend-secrets
```

The Helm chart only references the name:

```
existingSecretName: genai-backend-secrets
```

It does not need to know the secret values.

Readiness versus liveness

Readiness probe

Answers:

| Is this Pod ready to receive traffic?

A backend may be alive but not ready because:

- Startup is incomplete
- Database migrations are running
- Model configuration has not loaded
- Required connections have not initialized

When readiness fails, Kubernetes removes the Pod from Service traffic.

Liveness probe

Answers:

| Is the process still functioning?

If liveness repeatedly fails, Kubernetes restarts the container.

Avoid this mistake

Do not make the liveness endpoint depend on every external system.

For example, if Pinecone is temporarily unavailable, restarting every FastAPI Pod may make the incident worse.

A reasonable pattern is:

```
/health/live → process/event loop is alive
/health/ready → application can accept requests
/health/deps → detailed dependency diagnostics
```

9. Helm template example 2: Backend Service and HPA

```
# templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-backend
spec:
  type: ClusterIP

  selector:
    app: {{ .Release.Name }}-backend

  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}

---
{{- if .Values.autoscaling.enabled }}
# templates/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-backend
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-backend

  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization:
            {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}
```

Important HPA issue

When HPA is enabled, avoid hard-coding Deployment replicas as the only source of truth.

A common pattern is:

```
{{- if not .Values.autoscaling.enabled }}
replicas: {{ .Values.replicaCount }}
{{- end }}
```

This prevents a Helm upgrade from unnecessarily resetting the replica count being managed by HPA.

Also remember that resource-based HPA calculations depend on container resource requests.

Therefore, configure:

```
resources:
  requests:
    cpu: 500m
    memory: 1Gi
```

10. Frontend chart design

The frontend requires:

- Next.js Deployment
- ClusterIP Service
- Readiness and liveness probes
- Resource settings
- Optional HPA
- Ingress route for /

A simple production design lets the browser call a relative path:

```
fetch("/api/chat")
```

This is easier than embedding environment-specific backend domains into frontend code.

Request flow:

```
Browser request /api/chat
  |
  v
Same ALB and hostname
  |
  v
Ingress sends request to FastAPI Service
```

This also simplifies CORS because frontend and backend share the same public origin:

```
https://chat.example.com
```

Frontend values

```
replicaCount: 2

image:
  repository: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/genai-frontend
  tag: "2.1.0"
  pullPolicy: IfNotPresent

service:
  port: 80
```

```
targetPort: 3000

resources:
  requests:
    cpu: 100m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

11. Helm template example 3: Frontend workload and shared Ingress

```
# Frontend chart: templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-frontend
spec:
  replicas: {{ .Values.replicaCount }}

  selector:
    matchLabels:
      app: {{ .Release.Name }}-frontend

  template:
    metadata:
      labels:
        app: {{ .Release.Name }}-frontend

    spec:
      containers:
        - name: nextjs
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"

          ports:
            - name: http
              containerPort: 3000

          readinessProbe:
            httpGet:
              path: /
              port: http
              initialDelaySeconds: 5
              periodSeconds: 10

          livenessProbe:
            httpGet:
              path: /
              port: http
              initialDelaySeconds: 20
              periodSeconds: 20
```

```

        resources:
{{- toYaml .Values.resources | nindent 12 }}

---
# Frontend chart: templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-frontend
spec:
  type: ClusterIP

  selector:
    app: {{ .Release.Name }}-frontend

  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}

---
# Routing chart: templates/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: genai-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip

spec:
  ingressClassName: alb

  rules:
    - host: chat.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: genai-backend-prod-backend
                port:
                  number: 80

          - path: /
            pathType: Prefix
            backend:
              service:
                name: genai-frontend-prod-frontend
                port:
                  number: 80

```


Because `/api` is more specific than `/`, API traffic is directed to FastAPI and other traffic goes to Next.js.

Your FastAPI application should expose routes that match the forwarded paths:

```
@app.post("/api/chat")
async def chat(...):
    ...
```

Alternatively, introduce path rewriting through a compatible proxy or controller configuration. Directly supporting `/api` in FastAPI is usually simpler.

12. Environment-specific values

Keep reusable defaults in:

```
values.yaml
```

Then maintain environment overrides:

```
values-dev.yaml
values-stage.yaml
values-prod.yaml
```

Development

```
replicaCount: 1

image:
  tag: latest

config:
  appEnv: development
  logLevel: DEBUG
  topK: "3"

autoscaling:
  enabled: false
```

Production

```
replicaCount: 3

image:
  tag: "1.4.3"

config:
  appEnv: production
  logLevel: INFO
  topK: "8"

autoscaling:
```

```
enabled: true
minReplicas: 3
maxReplicas: 15
targetCPUUtilizationPercentage: 65
```

Install production:

```
helm upgrade --install genai-backend-prod \
  ./deploy/helm/genai-backend \
  --namespace genai-prod \
  --create-namespace \
  -f ./deploy/helm/genai-backend/values-prod.yaml
```

Install development:

```
helm upgrade --install genai-backend-dev \
  ./deploy/helm/genai-backend \
  --namespace genai-dev \
  --create-namespace \
  -f ./deploy/helm/genai-backend/values-dev.yaml
```

Same chart, different release and different values.

13. Helm releases

A **release** is an installed instance of a chart.

The same backend chart can produce multiple releases:

```
Chart: genai-backend

Releases:
├── genai-backend-dev
├── genai-backend-stage
└── genai-backend-prod
```

Each release has:

- Its own name
- Its own values
- Its own Kubernetes objects
- Its own revision history
- Its own rollback history

The Helm CLI includes commands for installing, upgrading, inspecting and rolling back releases. ([helm.sh](#))

14. Essential Helm commands

Create a chart

```
helm create genai-backend
```

Validate chart structure

```
helm lint ./genai-backend
```

Render templates locally

```
helm template genai-backend-prod \  
  ./genai-backend \  
  -f ./genai-backend/values-prod.yaml
```

This is extremely useful for debugging indentation and rendered names.

Server-side dry run

```
helm upgrade --install genai-backend-prod \  
  ./genai-backend \  
  --namespace genai-prod \  
  -f ./genai-backend/values-prod.yaml \  
  --dry-run
```

Install

```
helm install genai-backend-prod \  
  ./genai-backend \  
  --namespace genai-prod \  
  --create-namespace \  
  -f ./genai-backend/values-prod.yaml
```

Upgrade

```
helm upgrade genai-backend-prod \  
  ./genai-backend \  
  --namespace genai-prod \  
  -f ./genai-backend/values-prod.yaml \  
  --set image.tag=1.4.3
```

Recommended idempotent CI/CD command

```
helm upgrade --install genai-backend-prod \  
  ./genai-backend \  
  --namespace genai-prod \  
  --create-namespace \  
  -f ./genai-backend/values-prod.yaml \  
  --set image.tag="${IMAGE_TAG}" \  
  --wait \  
  --wait
```

```
--timeout 10m \  
--rollback-on-failure
```

Show releases

```
helm list --namespace genai-prod
```

Show revision history

```
helm history genai-backend-prod \  
  --namespace genai-prod
```

Roll back

```
helm rollback genai-backend-prod 3 \  
  --namespace genai-prod
```

This rolls the release back to revision 3.

15. Image-version pattern

Avoid deploying production images using:

```
tag: latest
```

Use immutable identifiers such as:

```
tag: "1.4.3"
```

or:

```
tag: "git-a37cb91"
```

CI/CD flow:

1. Developer merges code
2. CI runs tests
3. Docker image is built
4. Image is pushed to ECR
5. Pipeline obtains immutable image tag
6. Helm upgrade installs that tag
7. Readiness checks validate new Pods
8. Release succeeds or rolls back

Example:

```
helm upgrade --install genai-backend-prod \  
  ./genai-backend \  
  --set image.tag="${GIT_COMMIT_SHA}"
```

This makes deployments traceable to a particular commit.

16. Configuration and secrets flow

Non-sensitive flow

```
Git values-prod.yaml
  |
  v
Helm rendering
  |
  v
Kubernetes ConfigMap
  |
  v
FastAPI environment variables
```

Sensitive flow

```
AWS Secrets Manager
  |
  v
Terraform / External Secrets / CSI
  |
  v
Kubernetes Secret
  |
  v
Helm Deployment references Secret name
  |
  v
FastAPI environment variables
```

A clean division is:

Terraform owns

- VPC
- EKS
- RDS
- ElastiCache
- S3
- ECR
- IAM roles
- AWS Secrets Manager entries
- Kubernetes namespaces or shared infrastructure, optionally

Helm owns

- Application Deployments
- Application Services
- Ingress
- ConfigMaps
- HPA

- Pod disruption configuration
- Application service accounts

Secret synchronization system owns

- Copying or mounting secret values from AWS Secrets Manager into Pods
-

17. IAM access from FastAPI Pods

Avoid giving AWS permissions to every worker node and implicitly sharing them with all Pods.

Use an EKS Pod Identity or pod-level IAM mechanism so the FastAPI service account receives only the permissions it needs.

For example, the backend might need:

```
s3:GetObject
s3:PutObject
bedrock:InvokeModel
secretsmanager:GetSecretValue
```

The frontend normally should not need AWS API permissions.

Conceptual Deployment configuration:

```
spec:
  template:
    spec:
      serviceAccountName: genai-backend
```

This provides better separation than using long-lived AWS access keys inside a Kubernetes Secret.

18. Rolling updates

A standard Kubernetes Deployment uses a rolling strategy.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

Suppose the backend has three Pods:

```
Version 1:
v1 Pod
v1 Pod
v1 Pod
```

During deployment:

```
v1 Pod
v1 Pod
```

```
v1 Pod
v2 Pod
```

After the v2 Pod becomes ready:

```
v1 Pod
v1 Pod
v2 Pod
```

The process continues until:

```
v2 Pod
v2 Pod
v2 Pod
```

`maxUnavailable: 0` attempts to preserve the existing available replica count during rollout, while `maxSurge: 1` permits one additional Pod temporarily.

When rolling deployment is sufficient

Use it when:

- Application versions are backward compatible
- Database schema changes are compatible
- Pods are stateless
- Normal gradual replacement is acceptable

19. Blue/green deployment

Blue/green maintains two complete versions:

```
Blue Deployment → current production
Green Deployment → new candidate
```

Both have their own Pods:

```
genai-backend-blue
genai-backend-green
```

Initially, the Service selects blue:

```
selector:
  app: genai-backend
  slot: blue
```

After validation, switch it to green:

```
selector:
  app: genai-backend
  slot: green
```

Benefits:

- Very quick cutover
- Quick rollback
- New version can be tested before receiving production traffic

Trade-offs:

- Temporarily requires roughly double the capacity
- Database compatibility becomes important
- Active chat sessions and streaming requests need careful handling
- Background jobs must not be processed twice

For most initial capstone implementations:

Start with rolling updates. Explain blue/green conceptually in interviews and use it for higher-risk deployments.

20. GenAI-specific deployment considerations

Streaming responses

FastAPI chat endpoints may use Server-Sent Events or WebSockets.

Consider:

- ALB idle timeout
- Graceful Pod termination
- Long-lived connections
- Load balancing behaviour
- Client reconnection
- Pod shutdown period

Use a termination grace period:

```
spec:
  terminationGracePeriodSeconds: 60
```

The application should stop accepting new requests and allow in-flight generation streams to finish where possible.

Stateless backend Pods

Do not store chat history only in Pod memory.

Instead use:

```
PostgreSQL → durable conversations and messages
Redis      → temporary session and workflow state
S3         → uploaded source files
Vector DB  → embeddings and searchable chunks
```

This allows any backend Pod to handle the next request.

Background ingestion

Document ingestion should usually not run inside the web request for large files.

A more robust architecture is:

```
FastAPI receives upload
      |
      v
Stores document in S3
      |
      v
Adds ingestion job to queue
      |
      v
Worker Deployment processes job
      |
      v
Chunks, embeds and indexes document
```

The worker can have its own Helm Deployment and HPA.

Possible scaling metric:

```
Redis/SQS queue depth
```

Resource limits

Embedding libraries, tokenizers and document parsers can consume significant memory.

Too-low memory limits can result in:

```
OOMKilled
```

Start with measured requests and limits rather than arbitrary values.

For example:

```
resources:
  requests:
    cpu: 500m
    memory: 1Gi
  limits:
    cpu: "2"
    memory: 3Gi
```

Then tune using real workload measurements.

21. Common mistakes

1. Storing secrets in values.yaml

Bad:

```
openaiApiKey: sk-actual-key
```

Better:

```
existingSecretName: genai-backend-secrets
```

2. Missing selector-label alignment

This will break Service routing:

```
# Deployment
labels:
  app: backend

# Service
selector:
  app: fastapi
```

The labels must match.

3. Using the same health check for everything

Do not make liveness depend on every external provider.

Separate:

```
Live
Ready
Dependency diagnostics
```

4. Missing resource requests

Without realistic requests:

- Scheduling becomes unreliable
- HPA CPU percentages become less meaningful
- One workload can consume excessive node capacity

5. Using mutable image tags

Avoid latest in production. Use a version or commit SHA.

6. Running only one backend replica

One replica creates downtime risk during:

- Pod replacement
- Node maintenance
- Deployment
- Process crashes

Production should generally have at least two replicas where high availability is required.

7. Treating Helm as a secret manager

Helm templates configuration, but it should not be the source of sensitive production values.

8. Scaling only on CPU

An API waiting on LLM responses may have many active requests while CPU remains low.

Consider concurrency, queue depth or request metrics.

9. Breaking API routes through Ingress

If Ingress forwards `/api/chat`, FastAPI must either expose `/api/chat` or the routing layer must explicitly rewrite the path.

10. Deploying incompatible database migrations

A rolling update briefly runs old and new application versions together.

Database changes should follow backward-compatible expand-and-contract practices:

1. Add new schema
 2. Deploy compatible application
 3. Migrate data
 4. Remove old schema later
-

22. Complete deployment sequence

1. Terraform creates:
 - VPC
 - EKS
 - RDS
 - Redis
 - S3
 - ECR
 - IAM roles
 - Secret-management resources
 2. CI builds:
 - FastAPI image
 - Next.js image
 3. CI pushes images to ECR.
 4. Kubernetes secret integration makes credentials available.
 5. Helm installs backend:
 `genai-backend-prod`
-

6. Helm installs frontend:
genai-frontend-prod
7. Helm installs shared routing:
genai-routing-prod
8. AWS Load Balancer Controller observes Ingress.
9. AWS provisions the ALB.
10. Route 53 points the application hostname to the ALB.
11. Users access:
https://chat.example.com
12. Browser calls:
https://chat.example.com/api/chat

23. Interview-ready summary

A strong answer would sound like this:

I package the FastAPI RAG backend and Next.js frontend as separate Helm charts so they can be independently deployed, scaled and rolled back. Each chart contains a Deployment, ClusterIP Service, health probes and resource settings. Non-sensitive configuration is injected through ConfigMaps, while credentials are obtained from existing Kubernetes Secrets synchronized from AWS Secrets Manager. A shared Ingress uses the AWS Load Balancer Controller to provision an ALB, routing /api to FastAPI and / to Next.js. The backend uses rolling updates, multiple replicas and HPA, while remaining stateless by storing durable state in RDS, temporary state in Redis, source files in S3 and embeddings in a vector database.

24. Interview Q&A

1. What is the difference between a Pod and a Deployment?

A Pod runs one or more containers. A Deployment manages replicas of Pods, replaces failed Pods and controls application rollouts.

2. Why do we need a Service if an Ingress already exists?

Ingress defines external HTTP routing. A Service provides a stable internal endpoint and selects the changing set of backend Pods.

3. What is a Helm chart?

A Helm chart is a reusable package containing Kubernetes templates, default configuration values and chart metadata.

4. What is a Helm release?

A release is an installed instance of a chart. The same chart can be installed multiple times with different names, namespaces and values.

5. How do you handle dev, stage and production configuration?

Use the same chart with separate values files:

```
values-dev.yaml
values-stage.yaml
values-prod.yaml
```

Avoid copying the entire chart for each environment.

6. How should secrets be supplied to a FastAPI Pod?

Store secrets in a dedicated secrets manager, synchronize or mount them into Kubernetes, and reference the Secret from the Deployment. Do not commit actual secret values to Helm values files.

7. How does traffic reach FastAPI on EKS?

The client reaches an AWS ALB. The AWS Load Balancer Controller configures the ALB from a Kubernetes Ingress. The Ingress routes `/api` to a ClusterIP Service, which routes traffic to ready FastAPI Pods.

8. How would you scale a RAG backend?

Start with HPA and resource metrics, but evaluate custom metrics such as concurrent requests, request latency or queue depth because RAG services are often I/O-bound rather than purely CPU-bound.

9. What is the difference between readiness and liveness probes?

Readiness determines whether a Pod should receive traffic. Liveness determines whether Kubernetes should restart the container.

10. How does Helm rollback work?

Helm stores release revisions. `helm rollback <release> <revision>` reapplies a previous release configuration and Kubernetes manifest set.

11. Rolling update versus blue/green?

A rolling update gradually replaces old Pods with new Pods and uses less extra capacity. Blue/green runs two complete versions and switches traffic between them, making cutover and rollback faster but requiring more capacity.

12. Why use separate frontend and backend Helm releases?

They have different image versions, scaling requirements, resource needs and release schedules. Separate releases allow one component to be upgraded or rolled back without modifying the other.

25. Final mental model

Docker	Packages each application
ECR	Stores container images
Kubernetes Deployment	Runs and manages the containers
Kubernetes Service	Provides stable network access to Pods
Ingress	Routes external HTTP traffic
AWS Load Balancer Controller	Converts EKS Ingress into an AWS ALB
ConfigMap	Supplies non-sensitive configuration
Secret	Supplies sensitive configuration
HPA	Adjusts the number of Pods
Helm Chart	Templates all Kubernetes resources
Helm Release	Represents one installed environment/application instance
Terraform	Creates the AWS and EKS infrastructure beneath the application